Listas Encadeadas

Algoritmos e Estruturas de Dados

Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Lista Encadeada (Linked List)

Listas encadeadas utilizam os Nós como base para a sua construção.

Existem diversos tipos de listas encadeadas:

- Listas duplamente encadeadas são as mais populares
- A forma que as listas trabalham na memória é dependente do conceito de Nó

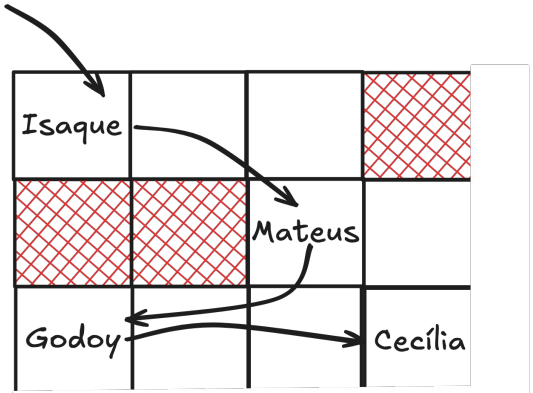
Com listas encadeadas reservamos o quarto no Hotel DDR5 sem nos preocuparmos em estar lado a lado uns com os outros.

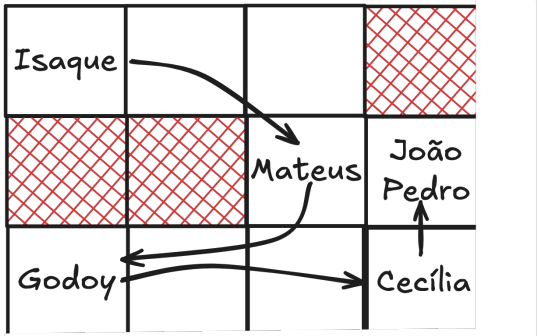
-> Porém, uma pessoa sabe onde fica o quarto de seu "melhor amigo".

Lista Encadeada (Linked List)

E se mais uma pessoa estiver interessada em aderir à viagem?

Lista Encadeada de interessados





Memória em uso (quarto ocupado)

Memória Livre (quarto vazio)

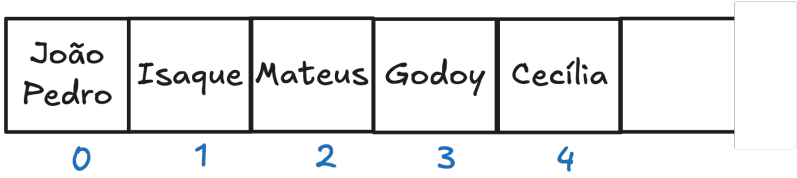
Memória em uso (quarto ocupado)

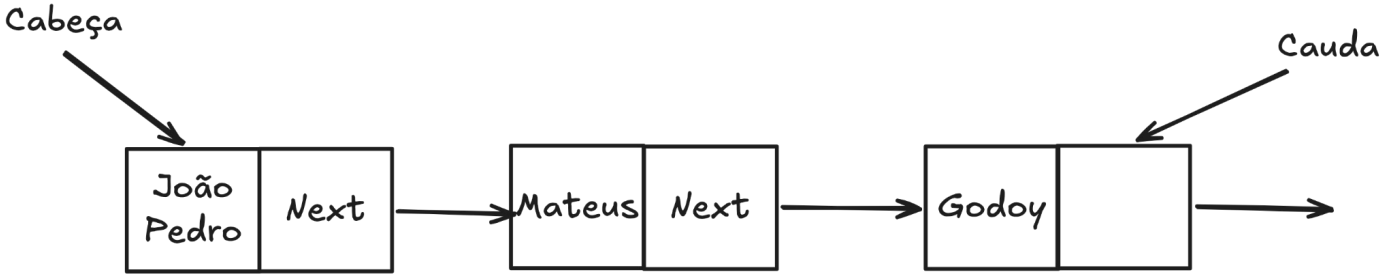
Memória Livre (quarto vazio)

carlos.melo@upe.br

Lista Encadeada (Linked List)

Array vs Linked Lists

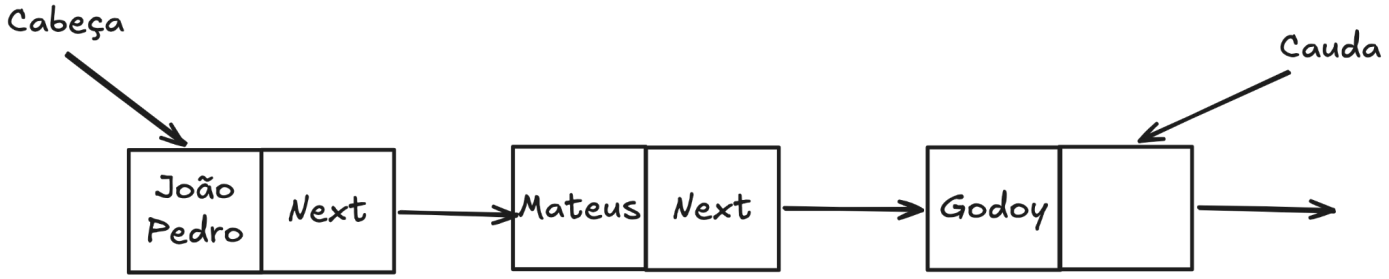




Array vs Linked Lists

Inicialização:

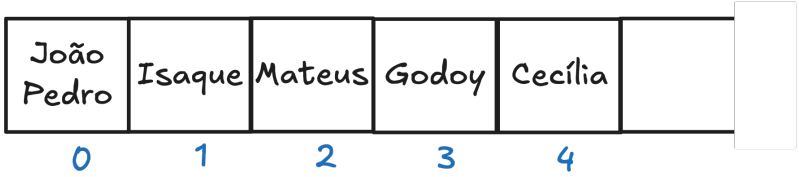
- Para os arrays em muitas linguagens de programação devemos definir o tamanho inicial do array, algo que pode gerar problemas caso não saibamos quantos elementos iremos armazenar, neste caso podemos alocar espaço em excesso ou em falta.
- Já para a lista encadeada, basta criarmos os primeiros nós, se não adicionarmos novos, não iremos precisar de espaço adicional e, não iremos desperdiçar espaço alocando mais do que precisaríamos.



Array vs Linked Lists

Busca e Acesso:

- Nos arrays sabemos exatamente a posição de cada elemento por meio de seu índice.
- Já na lista encadeada os elementos estão distribuídos em memória, não há acesso por índice e sempre que precisarmos de um elemento precisaremos percorrer a lista (ou parte dela) para encontrá-lo, uma operação que geralmente é mais custosa.



Array vs Linked Lists

Inserção

- Nos arrays temos três opções de inserção:
  - **O espaço em que iremos inserir está vazio** e não precisamos mover elementos subsequentes nem alocar mais espaço, inserção rápida (O(1))
  - **O espaço em que iremos inserir está ocupado**, não precisamos alocar mais espaço, mas precisamos mover elementos subsequentes, inserção lenta (O(n))
  - **Não há mais espaço para inserção no array**, precisamos criar um novo, copiar todos os dados do array já existente e então adicionar nosso elemento, inserção muito lenta (O(n)).

Array vs Linked Lists

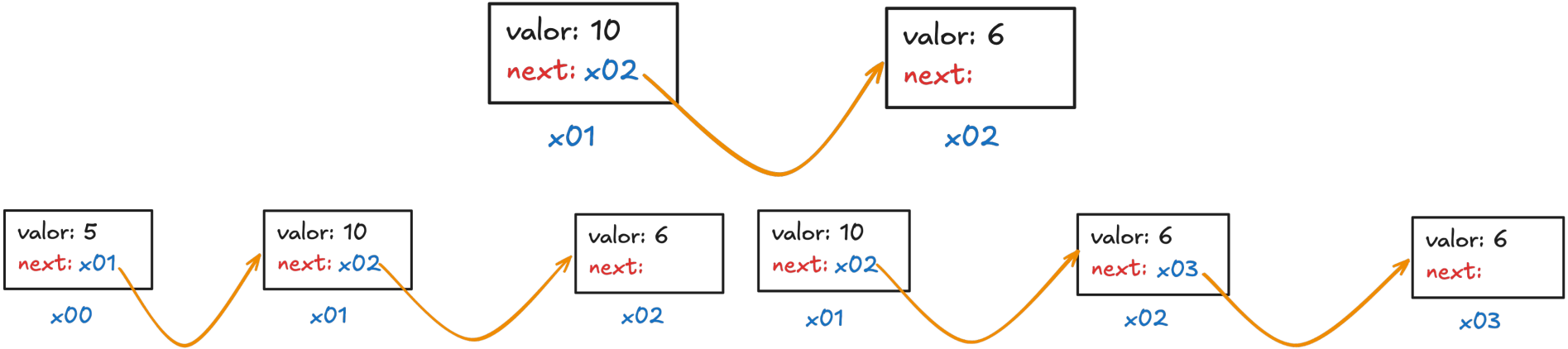
Inserção

- Já na lista encadeada as inserções geralmente são bastante rápidas, pois não é preciso mover elementos nem realocar mais armazenamento, porém precisamos percorrer a lista quando queremos inserir no meio.

Array vs Linked Lists

Inserção

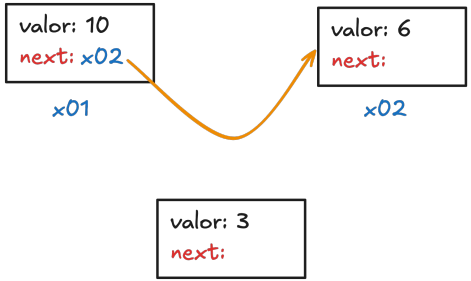
- Inserções no final e no início da lista encadeada podem ser feitas de forma especial por meio de elementos chamados cabeça e cauda (veremos em breve).

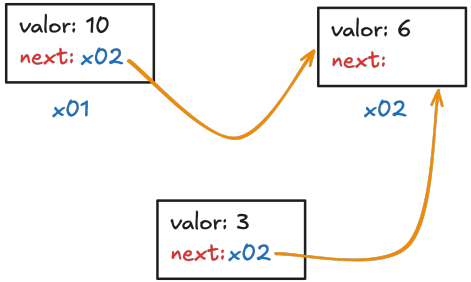


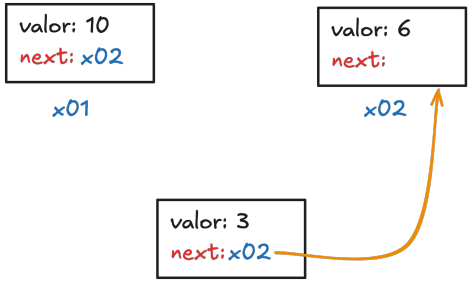
Array vs Linked Lists

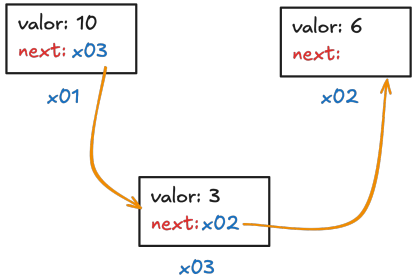
Inserção

- Inserções no meio demandam a realização de uma busca simples, mas sem movimentação de elementos subsequentes.



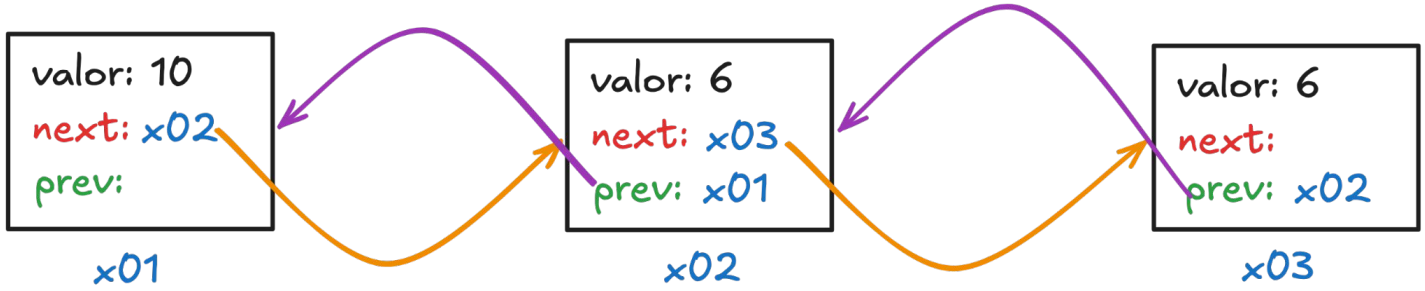






Doubly Linked Lists (Listas Duplamente Encadeadas)

Além do ponteiro para o próximo, precisamos apontar para o anterior.



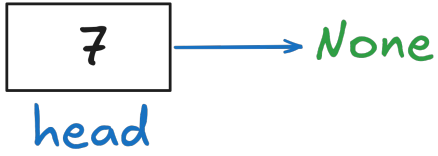
Linked Lists

Operações mais frequentes em listas:

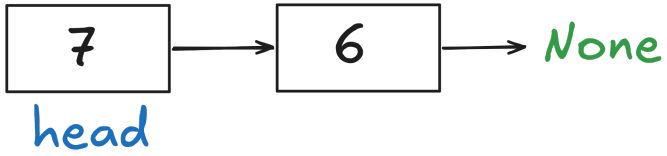
- `insert_first()` nos permite adicionar um **Nó** ao início da lista
- `insert_last()` nos permite adicionar um **Nó** ao final da lista
- `insert_at()` nos permite adicionar um **Nó** em qualquer posição
- `remove_first()` nos permite remover um **Nó** do início da lista
- `remove_last()` nos permite remover um **Nó** do final da lista
- `remove_at()` nos permite remover um **Nó** em qualquer posição da lista
- `get_element_at()` nos permite visualizar o **Nó** em qualquer posição da lista
- `is_empty()` nos permite identificar se existe ao menos um **Nó** na lista

Linked Lists

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```



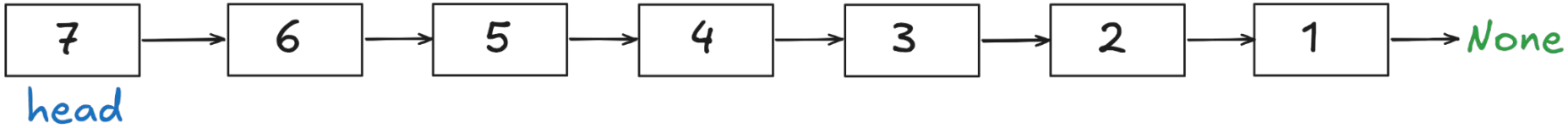

```python
head = Node(7)
head.next = Node(6)
```



-> Adicione mais 5 nós a lista

Linked Lists

```
head.next.next = Node(5)
head.next.next.next = Node(4)
head.next.next.next.next = Node(3)
head.next.next.next.next.next = Node(2)
head.next.next.next.next.next.next = Node(1)
```



-> Imprima os valores para cada nó da lista

Linked Lists

```
print(head.value)
print(head.next.value)
print(head.next.next.value)
print(head.next.next.next.value)
print(head.next.next.next.next.next.value)
print(head.next.next.next.next.next.next.value)
print(head.next.next.next.next.next.next.next.value)
```

-> É viável?

Exercício

1) Implemente uma função que atravessa toda a lista

```python
def print_list(head):
    no_atual = head

    while no_atual is not None:
        print(no_atual.value)
        no_atual = no_atual.next

print_list(head)
```

Exercício

2) Crie uma função que converta uma lista linear em Python para uma Lista Encadeada

```python
def creates_linked_list_from_list(lista):
    head = None
    tail = None

    for value in lista:
        if head is None:
            head = Node(value)
            tail = head
        else:
            tail.next = Node(value)
            tail = tail.next
    return head
```

carlos.melo@upe.br

Implementação de Linked List

Criando a classe da Lista Encadeada

```python
class LinkedList:
    def __init__(self) -> None:
        self.head = None
        self.tail = None
        self.length: int = 0

    def __str__(self) -> str:
        # O(n) - Percorre a lista para formar a string
        values = []
        current = self.head
        while current:
            values.append(str(current.value))
            current = current.next
        return f"LinkedList(len={self.length}, values=[{' -> '.join(values)}])"

    def __len__(self) -> int:
        # O(1) - Retorna o tamanho
        return self.length
```

Implementação de Linked List

Método para verificar se a lista está vazia

```
1    def is_empty(self) -> bool:
2        # O(1)
3        return not self.length
```

Implementação de Linked List

Criando o método de inserir no começo

```python
    def insert_first(self, value: Any) -> None:
        # O(1) - Apenas muda o ponteiro
        new_node = Node(value)
        new_node.next = self.head
        self.head = new_node
        if self.length == 0:
            self.tail = new_node
        self.length += 1
```

Implementação de Linked List

Criando o método de inserir no final

```python
    def insert_last(self, value: Any) -> None:
        # O(1) - com a causa é só ir direto pra ela
        if self.is_empty():
            return self.insert_first(value)

        new_node = Node(value)
        self.tail.next = new_node
        self.tail = new_node
        self.length += 1
```

Implementação de Linked List

Criando o método de inserir na posição

```python
def insert_at(self, value: Any, position: int) -> None:
    # O(n) - No pior caso percorre n passos
    if position <= 0:
        return self.insert_first(value)
    if position >= self.length:
        return self.insert_last(value)

    current = self.head
    for _ in range(position - 1):
        current = current.next

    new_node = Node(value)
    new_node.next = current.next
    current.next = new_node
    self.length += 1
```

Implementação de Linked List

Criando o método de remover no início

```python
def remove_first(self):
    # O(1)
    if self.is_empty():
        return None

    removed = self.head
    self.head = self.head.next
    self.length -= 1

    if self.length == 0:
        self.tail = None

    removed.next = None
    return removed
```

carlos.melo@upe.br

Implementação de Linked List

Criando o método de remover no final

```python
    def remove_last(self):
        # O(n) - Precisamos do nó anterior ao tail para desconectar
        if self.length <= 1:
            return self.remove_first()

        current = self.head
        while current.next.next:
            current = current.next

        removed = current.next
        current.next = None
        self.tail = current
        self.length -= 1
        return removed
```

Implementação de Linked List

Criando o método de remover na posição

```python
def remove_at(self, position: int):
        # O(n) precisa percorrer
        if position < 0 or position >= self.length:
            return None

        if position == 0:
            return self.remove_first()

        if position == self.length - 1:
            return self.remove_last()

        current = self.head
        for _ in range(position - 1):
            current = current.next

        removed_node = current.next
        current.next = removed_node.next
        removed_node.next = None

        self.length -= 1
        return removed_node
```

carlos.melo@upe.br

Exercício

Implemente a lista duplamente encadeada com base no código da lista encadeada

Dicas:

- Cada nó terá dois ponteiros (next e prev)
- Os métodos que atualizam os ponteiros precisarão ser readaptados:
  - insert_first
  - insert_last
  - remove_last
  - remove_at

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS